TRAVEL MATE

Offline-First Intelligent Mobility & Journey Planning Platform

Project Report

Team: Reabrock Coders

Problem statement	PS1 — The Last Mile Paradox: Accessible Mobility Under Uncertainty
Application	Travel Mate
Primary interfaces	ROUTE INFO; COST AND DEPARTURE ESTIMATION
Deployment target	Local/offline-capable Node.js application with local data storage
Report date	6 November 2026

Prepared from the supplied project presentation, application README/source files, stated challenge constraints, and the setup/test evidence shared during development.

1. Executive Summary

Travel Mate is a mobility-planning application designed around the idea that the shortest route is not always the most usable route. It aims to help travellers compare locally available transport options using safety-related indicators, accessibility, lighting, community activity, fare, walking distance and journey duration. A second interface estimates when a traveller should leave home to reach a selected transport stop and arrive before a requested deadline, while considering an early-arrival buffer and available public-amenity records.

The challenge requires the essential journey to continue when external internet connectivity and third-party APIs are unavailable. It also requires the application to withstand malformed or malicious input data. The current implementation is structured as an offline-first local web application: a browser interface, a Node.js/Express backend, local dataset storage, optional local MongoDB persistence, and optional localhost-only Ollama evaluation.

The supplied project documentation explicitly identifies its map as a bundled schematic illustration rather than a live Leaflet/OpenStreetMap map. This is an intentional offline-constraint choice: the current interface does not fetch online map tiles, geocoding, routing services, CDNs or external APIs. A fully offline Leaflet map would require locally bundled Leaflet assets and pre-downloaded map tiles/route geometry; those should not be described as currently implemented.

The application was started successfully in the development environment and the terminal showed both “Travel Mate running at http://localhost:5000” and “MongoDB connected”. MongoDB Compass also displayed a successful connection and the travel_mate database. These observations verify service connectivity, but a real journey write/read test is still required to verify end-to-end persistence.

2. Problem Statement and Objectives

PS1 — The Last Mile Paradox: Accessible Mobility Under Uncertainty. A route that appears shortest on a map may not be usable by a traveller because of safety, accessibility, closures, cost, delays or limited public facilities.

Key challenges identified in the presentation:

- **Route vulnerability:** routes can become unsafe or unavailable because of closures, missing ramps, accidents or theft-prone areas.
- **Cost–time trade-off:** variable fares and delays make it difficult to choose an affordable route and calculate when to leave.

Project objectives:

- Compare transport choices using more than distance alone.
- Present an indicative route score on a 0–10 scale, with a transparent explanation.
- Identify an economical option subject to the supplied local fare data and user budget.
- Calculate a leave-home estimate from a timetable departure, access-to-stop time and early-arrival buffer.
- Show locally recorded public amenities and clearly distinguish unknown information from confirmed availability.
- Continue essential functions without internet access or third-party APIs.
- Validate imported and submitted data before accepting it.

3. Application Interfaces and User Journey

3.1 ROUTE INFO

The ROUTE INFO interface filters locally stored routes by origin, destination, step-free accessibility and maximum walking distance. Matching routes are ranked using a deterministic weighted score. The implementation uses these factors: safety (40%), accessibility (20%), lighting (15%), community activity (10%), fare (10%) and duration (5%). The

score is an indicative ranking based on supplied data; it is not a live safety prediction.

3.2 COST AND DEPARTURE ESTIMATION

The traveller supplies an origin, destination, desired arrival time, time needed to reach the stop, early-arrival buffer, maximum budget and waiting preference. The local timetable is checked for a departure that can satisfy the arrival deadline. The application calculates a recommended time to leave home from the selected departure and the user-provided access time and buffer. If no local route or suitable future departure exists, the interface reports that no estimate can be produced from the available dataset.

3.3 Public amenity check

Amenity records are looked up from the local dataset for the destination. The application reports requested facilities that are recorded as available and separately lists facilities not recorded. Missing amenity information is not treated as proof that a facility exists.

3.4 Journey history

The MongoDB journey-save update adds a POST /api/journeys operation and a GET /api/journeys operation. A saved record includes origin, destination, arrival deadline, access time, early-arrival buffer, optional budget, waiting preference and the selected transport option with departure, leave-home and estimated-arrival timestamps. MongoDB mode stores these records in an additive journeys collection; the existing Route and Report schemas are retained.

4. Technical Architecture

Layer	Technology / responsibility
Frontend	HTML, CSS and browser JavaScript; two main interfaces; local form validation and result rendering.
Backend	Node.js and Express; local REST endpoints; independent server-side validation; static asset serving.
Database	MongoDB via Mongoose when MONGO_URI is configured and reachable; local JSON-file persistence as a fallback.
Map presentation	Bundled schematic SVG/map illustration. No live Leaflet/OSM tile or routing requests in the current offline build.
Optional local AI	Ollama on 127.0.0.1 only when a local model is configured. Deterministic scoring remains the fallback.
Data import/export	Local JSON/CSV dataset import, validation, export and reset capabilities.

Core local endpoints documented or present in the application include /api/health, /api/routes, /api/reports, /api/amenities, /api/dataset/import, /api/assistant/evaluate, POST /api/journeys and GET /api/journeys.

MongoDB connection used during local setup: mongodb://127.0.0.1:27017/travel_mate. The Node server uses port 5000 by default.

5. Compliance with Challenge Constraint 1 — No External Connectivity

Constraint: external internet connectivity and third-party API access become unavailable during the challenge. The application must continue its essential user journey using information and components available within the local environment.

- Frontend assets are served locally by the Express application.

- No external map tiles, online geocoder, online routing service, CDN, font host or cloud API is required by the documented offline build.
- Routes, schedules and amenity information are supplied through a local dataset.
- Local JSON persistence supports operation when MongoDB is unavailable.
- MongoDB can be run locally and accessed through localhost.
- Optional LLM evaluation is restricted to a local Ollama service; the application does not call a cloud LLM.
- The app can continue its core route ranking and departure calculation using deterministic local logic if Ollama is not configured.

Important map scope: The challenge presentation lists Leaflet.js and OpenStreetMap in its proposed tech stack. However, the supplied implementation README says the current map is a bundled schematic illustration, not a live Leaflet/OSM map. Live OSM tiles require internet. If a geographic map is required during the offline challenge, package Leaflet JavaScript/CSS and the required tile set/route geometry locally in advance.

6. Compliance with Challenge Constraint 4 — Malformed or Malicious Inputs

Constraint: the application may receive missing fields, duplicates, invalid timestamps, negative or unrealistic values, unexpected Unicode, malformed JSON/CSV or unexpected data types.

The supplied implementation documentation and backend code describe the following defensive controls:

- JSON request bodies are limited to 1 MB.
- Server-side validators check required text fields, types, allowed ranges and array shapes.
- Departure times are checked against the HH:MM format; invalid values are rejected.
- Numeric fields are checked for finite values and permitted ranges, including fare, duration, distance, walk distance and 0–10 indicators.
- Boolean fields and report severity values are checked against accepted types/values.
- Text is Unicode-normalized, control characters are removed and lengths are bounded.
- CSV parsing checks quoting, duplicate headers, required columns and row column counts.
- JSON parsing errors and malformed imports are rejected.
- Dataset import is validated before replacing the existing local dataset.
- Write endpoints have a per-client rate limit; security response headers are set.

These controls reduce malformed-input risk but do not replace a full security review. A production deployment should additionally use authentication/authorization where needed, audit logging, backups, dependency updates, operational monitoring and tested recovery procedures.

7. Route Evaluation and Local AI

The deterministic route score provides a stable baseline. It combines the locally supplied safety, accessibility, lighting and activity indicators with fare and duration factors. The local LLM is optional and can return a separate 0–10 assessment and short rationale through Ollama running on localhost. If the model is unavailable or returns invalid output, the deterministic score remains the ranking fallback.

The model cannot establish live safety conditions or predict incidents without reliable and current historical/contextual data. Therefore, scores should be presented as indicative evaluations of the supplied dataset, not guarantees of safety. No score should be described as a validated crime-risk prediction.

8. Data Sources, Assumptions and Limitations

The project README explicitly states that the included Chennai routes and departure times are illustrative demo data. They are not verified current timetables, live traffic, current fares or measured safety statistics. The departure engine calculates from the locally stored timetable and the traveller's entered access time and buffer; it does not know real-time delays.

A real-world rollout requires maintained and permissioned datasets for transport operator timetables, fare tables, route geometry, facility inventories, accessibility attributes and safety/community observations. The data should have source, timestamp, geographic coverage and verification metadata. Stale or missing data should be surfaced clearly.

The current offline map is schematic. A locally bundled geographic map and route geometry are future implementation work if the project needs map-level navigation without internet. Leaflet alone does not provide offline map data; the map assets/tiles must also be available locally.

A public deployment differs from the isolated challenge environment. MongoDB and any persistent JSON storage must be hosted with durable storage and backups. A deployment platform with ephemeral filesystem storage can lose local JSON records on restart unless a persistent volume is attached. Strict no-internet operation also requires infrastructure-level egress restrictions and local copies of all runtime dependencies.

9. Setup and Verification

9.1 Local startup

The development setup shared during testing used Windows PowerShell, Node.js and MongoDB Community Server. The MongoDB service was reachable on localhost port 27017. A port conflict on application port 5000 was identified and resolved by locating the owning process before restarting the server.

Example PowerShell startup commands:

```
$env:MONGO_URI="mongodb://127.0.0.1:27017/travel_mate"
npm install
npm start
# Open http://localhost:5000
```

9.2 Evidence observed during development

- The Node application printed "Travel Mate running at http://localhost:5000".
- The backend printed "MongoDB connected".
- MongoDB Compass displayed "Connected to Travel mate" and showed the travel_mate database.

9.3 Required end-to-end verification before claiming persistence

- Submit a valid Cost and Departure Estimation request in the browser.
- Confirm the interface reports that the journey was saved and identifies the storage mode.
- Call GET /api/journeys or refresh the journey history in the interface.
- Refresh MongoDB Compass and inspect travel_mate → journeys for the new document.
- Restart the application and confirm the record remains available.
- Repeat with malformed/missing fields and verify that the API rejects the request without corrupting existing records.

The shared screenshots establish successful server startup and database connection, but they do not by themselves prove a successful journey insert, retrieval or persistence after restart. Those checks remain acceptance tests.

10. Future Scope

- **AI-powered routing:** improve personalized recommendations using validated historical data and locally available machine-learning models.
- **Real-time alerts:** incorporate locally cached or preloaded incident/closure updates when connectivity is unavailable.
- **Community reporting:** let travellers report unsafe areas and accessibility issues, with moderation and verification.
- **Smart transport integration:** import verified bus, metro, train and other operator schedules and fare data.
- **Offline geographic mapping:** bundle Leaflet assets, offline map tiles and route geometry for the intended service area.
- **Operational readiness:** authentication, role-based access, audit trails, backup/restore, monitoring and deployment hardening.

11. Conclusion

Travel Mate addresses the Last Mile Paradox by combining route comparison, indicative safety/accessibility scoring, cost-aware transport selection, leave-home estimation and local public-amenity information. Its offline-first design is aligned with the challenge requirement to keep essential functions available without external services, while its validation controls address malformed and potentially malicious datasets.

The strongest next step is to complete and document end-to-end acceptance testing: submit a journey, verify the new MongoDB document, retrieve it through the API/UI, and confirm it survives an application restart. The project should also keep its map and data limitations explicit: the current map is schematic, and the bundled timetable/route data is illustrative until replaced with verified datasets.

Source basis: “Reabrock Coders” Travel Mate presentation PDF; Travel Mate Offline-first mobility application README; supplied Node.js/Express backend and browser JavaScript; challenge constraint cards; and setup screenshots shared in this conversation. Claims about current implementation are limited to what those sources show.